

KPI Intelligence-to-Action Engine

A system that watches business KPIs, figures out what actually moved them, and tells you what to do about it. Or admits it doesn't know, when the data doesn't support a confident answer.

Accenture Hackathon — Round 2, Problem Track 3 | Live: <https://kpi-engine.vercel.app> | GitHub: github.com/Vapa1510/accenture_businessintelli

It was built around NovaMart, a synthetic e-commerce business, but the pipeline underneath isn't tied to that dataset. Point it at real transactions, marketing spend, and external signals, and the same engine keeps working.

Why this exists

Most “AI analytics” tools have the same weak spot: they let a language model touch the numbers. Ask it why revenue dropped and it will happily give you a confident, well-written, occasionally made-up answer. That's fine for a demo and dangerous for a finance meeting.

This project takes the opposite stance. Every number you see — the percentage change, the driver breakdown, the confidence score — is computed in plain Python using pandas, statsmodels, and scikit-learn. The LLM is only ever allowed to turn those numbers into readable prose, and even then, its output gets checked line by line against the underlying evidence before it's shown to anyone. If it invents a figure, that response is thrown out and a template-based explanation is used instead. The user never sees a hallucinated number, and the request never fails.

The other core idea is that “I don't know” is a valid answer. If the evidence is thin, contradictory, or too recent to trust, the engine says so instead of guessing. That's arguably the more interesting part of this project.

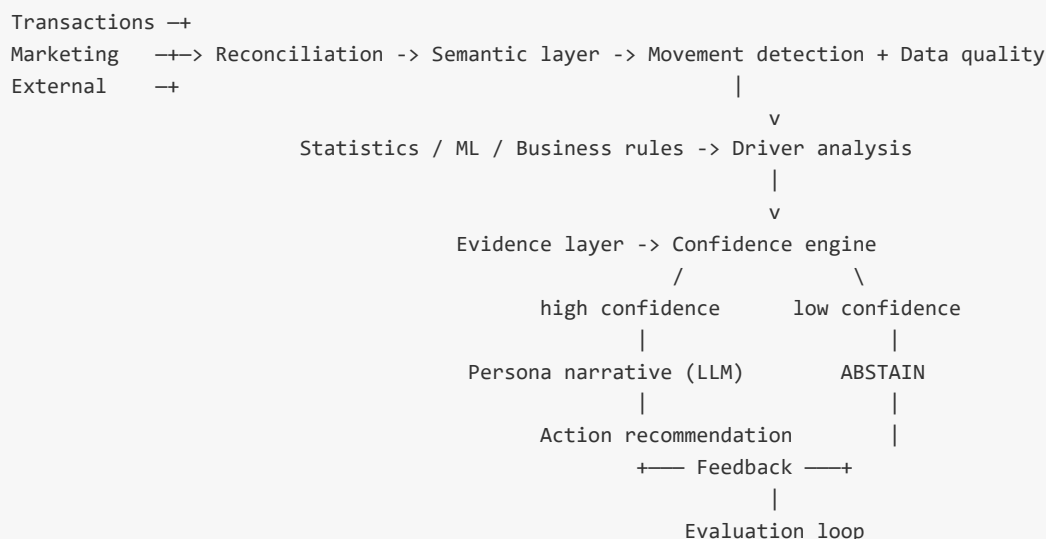
What it actually does

- Detects meaningful KPI movements (revenue, orders, AOV, margin, conversion) against a rolling baseline
- Splits a revenue change into “how much came from volume vs. how much came from price/mix” using LMDI decomposition, which sums exactly to the total change with no leftover residual to hand-wave away
- Fits a regression against marketing spend, conversion, price, and supply availability to estimate which levers actually moved the needle, in dollar terms
- Flags anomalous days with IsolationForest, using multiple signals at once rather than a single threshold
- Scores its own confidence using five weighted signals (completeness, freshness, statistical strength, cross-source agreement, historical coverage) and abstains outright below a threshold, or when specific red flags show up (stale marketing data, sub-30-day history, conflicting sources)
- Writes a narrative in plain language, tailored to who's asking. An executive gets a different summary than an analyst

- Recommends an action, with an approval workflow gated by role
- Lets you simulate additional days of data with adjustable marketing spend, stockout rate, and competitor pricing, to see how the engine reacts
- Logs feedback on its own recommendations and factors that feedback into future driver attribution

How it's put together

Data comes in from three sources (transactions, marketing, and external signals like competitor pricing, weather, and supply) and gets reconciled against a shared semantic layer, then flows through movement detection and data-quality checks in parallel. From there, statistics, a bit of ML, and hand-written business rules all contribute to driver analysis. Everything gets bundled into an evidence object with full lineage, and a confidence engine decides what happens next: high confidence goes to the narrative generator and an action recommendation, low confidence stops the process and returns an abstention with a reason. Either way, the outcome can be rated by the person using it, and that feedback loops back into the system.



Why these particular tools

A few choices worth explaining up front. The full reasoning, including alternatives we considered and rejected, is in `DESIGN.md`.

LMDI over Shapley for revenue decomposition. Revenue is $\text{Orders} \times \text{AOV}$, and we wanted the volume effect and the price/mix effect to add up exactly to the total change. Shapley is more theoretically elegant for many-factor problems, but it leaves a residual that’s hard to explain to someone reading a dashboard. LMDI doesn’t.

OLS regression, not causal inference. We fit a regression and report standardized coefficients, R^2 , and p-values. We do not claim causality. There’s no instrument, no natural experiment, and pretending otherwise with synthetic e-commerce data would be misleading. The UI and the narrative validator both enforce “associational, not causal” language.

IsolationForest for anomaly detection, because a single day can look fine on any one metric but abnormal once you consider revenue, conversion, and marketing spend together. Z-score thresholds miss that, and a full autoencoder is more than this data needs.

Templates by default, LLM as an opt-in upgrade. `narrate()` produces prose from Python templates driven entirely by computed values, so no infrastructure or API key is required. If you set `ANTHROPIC_API_KEY` and request `?live=true`, the structured evidence (never raw data) gets sent to Claude, and the response is scanned for every number it contains. Anything that doesn’t match the evidence within a small tolerance gets rejected, and the template output is used instead.

Stack

Layer	Tools	Notes
API	FastAPI	async, auto-generated docs at <code>/docs</code>
Analytics engine	pandas, numpy, scipy, statsmodels, scikit-learn	all the actual number-crunching lives here
Database	PostgreSQL via SQLAlchemy	falls back to SQLite automatically if Postgres isn’t available
Cache & cost routing	Redis, with an in-memory fallback	also classifies each request so cheap lookups skip the LLM entirely
Auth	JWT + role-based access control	redaction happens server-side, not in the UI
Frontend	React, TypeScript, Tailwind, Recharts	
Narrative (optional)	Claude API	off unless you provide a key

Getting started

You've got two options: Docker (gives you Postgres and Redis for free) or running it manually against SQLite. Both work fine for trying this out.

Option 1: Docker

```
cp .env.example .env
docker compose up --build
```

- App: `http://localhost:5173`
- API docs: `http://localhost:8000/docs`

The backend seeds all four demo scenarios into Postgres automatically on startup.

Option 2: Run it directly

No Postgres, no Redis needed. SQLite and an in-memory cache take over automatically.

```
# backend
cd backend
python -m venv .venv
source .venv/bin/activate          # on Windows: .venv\Scripts\activate
pip install -r requirements.txt
python -m app.seed
uvicorn app.main:app --reload --port 8000

# frontend (second terminal)
cd frontend
npm install
npm run dev
```

Logging in

There's no real signup flow, this is a demo. Every role name doubles as its own username and password:

Username	Role	What they can see	What they can approve
executive	Executive	everything, including customer-level detail	budget, strategic, campaign, inventory
marketing	Marketing Manager	revenue, orders, conversion, AOV	campaign, budget
operations	Operations Manager	revenue, orders, margin	inventory, supplier
analyst	Analyst	everything, deepest evidence view	nothing, read-only by design

Switching roles in the UI re-authenticates you as that role. Worth noting: the redaction isn't a frontend trick. Restricted KPIs are stripped out on the server before the response ever leaves the API, so there's no way to see them by poking at the network tab.

The four demo scenarios

Data is generated with a seeded random number generator, so re-running a scenario always produces identical numbers. Useful for testing, and for not looking silly during a demo.

Scenario	What's happening under the hood	What the engine should do
Revenue Decline	Marketing pullback plus a stockout in the North region	Explain it: revenue down ~14.4%, marketing surfaces as the top driver
Revenue Growth	A campaign lift combined with a shift toward premium products	Explain it: revenue up ~21.4%
Contradictory Evidence	Marketing data is 29 hours stale and 18% of external rows are missing	Abstain — the signals don't agree with each other
New Product	A product launched only 18 days ago	Abstain — not enough history to establish a baseline

You can also step through time yourself using the simulator, which lets you add another day of data and dial marketing spend, stockout rate, and competitor pricing up or down to see how the engine's read on the situation changes.

API reference

Full interactive documentation lives at [/docs](#) once the backend is running. Quick summary:

Route	What it does
POST <code>/api/auth/token</code>	Get a JWT for a given role
GET <code>/api/scenarios</code>	List available scenarios and roles
GET <code>/api/insight</code>	The full insight: computed values, narrative, validation result, telemetry
GET <code>/api/drivers</code>	Regression output, correlations, anomaly flags, drill-down data
GET <code>/api/sources</code>	Freshness, row counts, and schemas for each data source
GET <code>/api/semantic</code>	The KPI contract definitions
POST <code>/api/chat</code>	Ask a question in plain language; routed to a structured engine query
POST <code>/api/feedback</code> / GET <code>/api/feedback</code>	Submit or view aggregated feedback on recommendations
GET <code>/api/health</code>	Latency, cache hit rate, how often the LLM was avoided, abstention rate
POST <code>/api/scenarios/{id}/simulate-day</code>	Append a new simulated day to a scenario
POST <code>/api/scenarios/{id}/reset</code>	Wipe a scenario back to its seeded state

Running the tests

```
cd backend
pytest -q
```

This runs 39 tests split across two files. The ones in `test_engine.py` matter most. Each scenario has a known, deliberately injected ground truth, so the tests check that the engine actually *recovers* that truth rather than just producing some plausible-looking output. Specifically, they check things like:

- Revenue declines by 10 to 20% and marketing shows up in the top two attributed drivers
- LMDI effects sum to the total revenue change within a dollar
- The contradictory scenario abstains and cites stale marketing data as the reason
- The new-product scenario abstains because of sparse history
- No `NaN` or infinity values leak out anywhere
- Every number quoted in every persona's narrative can be traced back to an evidence object

`test_api.py` covers the HTTP layer and auth/RBAC behavior, and `test_simulation.py` covers the day-by-day simulator.

Project layout

```
backend/  
  app/  
    engine/  
      generator.py    synthetic data generation (seeded, reproducible)  
      semantic.py     KPI contracts: definitions, thresholds, access rules  
      analytics.py    movement detection, LMDI, regression, anomaly detection  
      insight.py      evidence assembly and confidence scoring  
      narrative.py     template narratives + optional validated LLM narratives  
    main.py           FastAPI app and all routes  
    auth.py           JWT auth and role-based access control  
    db.py             SQLAlchemy models, Postgres/SQLite handling  
    cache.py          Redis/in-memory caching and cost-tier routing  
    tests/            pytest suite  
  
frontend/  
  src/  
    components/pages/ one file per screen (Overview, Drivers, Sources, Chat, Simulator, etc.)  
    api.ts             API client  
    App.tsx            routing and layout  
  
DESIGN.md             the reasoning behind each technical choice, in more depth
```

Honest limitations

Worth being upfront about a few things:

- The data is synthetic. If you want to point this at real numbers, swap out `generate()` in `generator.py` for a loader that returns the same three DataFrames. The rest of the pipeline doesn't need to change.
- The regression is associational, not causal, and both the UI and the narrative validator are built to stop causal-sounding language from slipping through.
- Feedback is collected and does influence future driver weighting, but nothing is retrained live, and the product doesn't pretend otherwise.
- The demo login setup uses the role name as its own password. That's fine for a prototype; replace `DEMO_USERS` in `auth.py` with a real identity provider before this goes anywhere near actual users.
- In local dev mode (`REQUIRE_AUTH=false`), unauthenticated requests default to executive access, purely to make iterating faster. Flip this before deploying anywhere that isn't your own laptop.

What we'd change with more time

- Swap OLS for something closer to actual causal inference (instrumental variables or diff-in-diff) once there's richer data to support it. The attribution function is already isolated enough to make this a contained change.
- Learn the confidence-score weights from analyst feedback instead of hand-tuning them.

- Make the chat endpoint stream responses instead of doing a full round trip.
- Build real data connectors once there's an actual source system to connect to.